EICTA Second Project Delivery – Neo4j and BPMN

Group Number: 4

Academic Year 2025–2026

Authors:

Annalaura Catalfo — Bocconi: 3200742 — PoliMI: 11182469
Guendalina Nardi - Bocconi: 3376114 — PoliMI: 11179726
Giacomo Nigro — Bocconi: 3186734 — PoliMI: 11182213
Sofia Varotto — Bocconi: 3221388 — PoliMI: 11182433

Contents

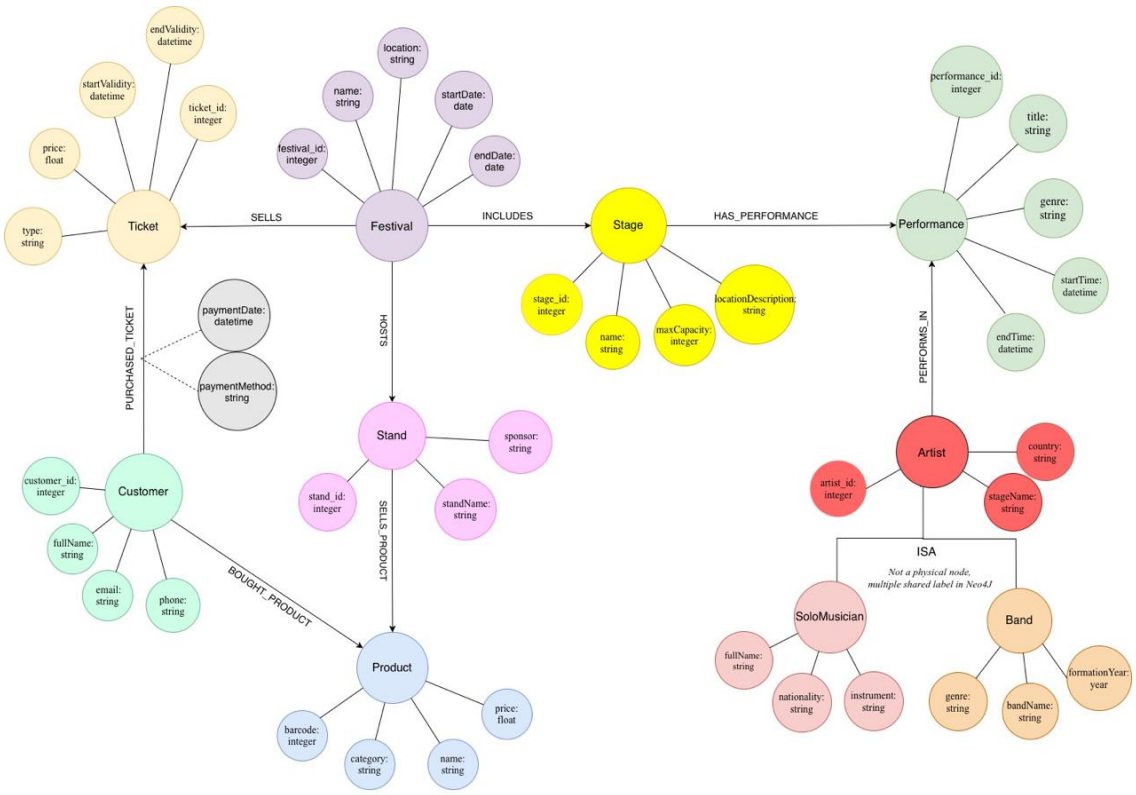
1 Neo4j **1**
1.1 Schematic representation of the chosen nodes 1
1.2 Nodes and relationships – Attributes and descriptions 2
1.2.1 Additional modeling assumptions 2
1.2.2 Nodes : labels, identifiers, properties 2
1.2.3 Relationships : Definition and semantics 3
1.2.4 Data loading and database adaptation 4
1.3 Cypher queries 6
1.4 Create / Update / Delete Queries 6
1.4.1 1) CREATE – Aurora Luna's debut performance 6
1.4.2 2) CREATE – New Stage "Eco Dome Milano" 7
1.4.3 3) UPDATE – VIP Ticket Price Increase 7
1.4.4 4) UPDATE – Vegetarian Sandwich on Sale 8
1.4.5 5) DELETE – Remove Outdated Festivals (Before 31 Dec 2025) 8
1.4.6 6) Italian Artists performing at Milano Soundwave 10
1.4.7 7) Electronic Performances after 20 June 2025 10
1.4.8 8) Top 5 Most Active Stands 11
1.4.9 9) Ticket Economy at Roma Indie Fest 12
1.4.10 Query 9 Output 12
1.4.11 10) Food & Drink Fans 12
1.4.12 11) Frequent Buyers 13
1.4.13 12) Festival Tourers 13
1.4.14 13) Longest Performance per Festival 14
1.4.15 14) Chasing Artistic Connections: Variable-Lenght Path Between Artists 15
1.4.16 15) Closest Connection: Shortest Path Between Two Artists . 16

2 BPMN **18**
2.1 BPMN diagram representation 18
2.1.1 Designed diagram 18
2.2 BPMN & Euphoric Events 22
2.2.1 Designed diagram 22

Chapter 1

Neo4j

1.1 Schematic representation of the chosen nodes



1.2 Nodes and relationships – Attributes and descriptions

1.2.1 Additional modeling assumptions

All assumptions from the first project are retained unless explicitly modified; for example, the removal of the Product ISA hierarchy and the Neo4j multi-label implementation of the Artist ISA hierarchy.

- **Handling IDs.** Although Neo4j automatically assigns internal identifiers to nodes, we maintain explicit logical IDs as node properties to ensure consistency across imports and alignment with the original relational model. Where appropriate, identifiers originally stored as strings in SQL were converted to integers to keep data types consistent across the graph model.
- **Artist ISA Hierarchy.** In the original relational model, `Artist` was defined through a Total and Exclusive (T.E.) ISA hierarchy, distinguishing between `SoloMusician` and `Band` subtypes, each in a separate table (`ID_Solo_Musician`, `ID_Band`). The superclass `Artist` was conceptual, grouping common attributes such as `stageName`, `country`, and `genre`.

  In Neo4j, this hierarchy is modeled using a multi-label approach: each artist is a single node labeled either `:Artist:SoloMusician` or `:Artist:Band`. This preserves the T.E. semantics (each artist belongs to exactly one subtype), allows querying all artists via the shared label `:Artist`, and keeps subtype-specific attributes on the secondary label. A unified identifier `artist_id` replaces the two SQL subtype IDs, enabling a single uniqueness constraint on `:Artist(artist_id)`. Common attributes (`stageName`, `country`, `genre`) are stored on every artist node; subtype-specific attributes (e.g., `instrument`, `nationality`, `formationYear`, `bandName`) remain on the respective subtype.
- **Removal of Product ISA Hierarchy.** In the relational model, `Product` used an ISA hierarchy with three subtypes (`Food`, `Drink`, `Merchandise`). In Neo4j, the hierarchy is flattened into a single label `:Product`. Each product is uniquely identified by its `barcode` and differentiated by a `category` attribute with values "Food", "Drink", or "Merchandise".

1.2.2 Nodes : labels, identifiers, properties

- **Festival** — `:Festival`
  Identifier: `festival_id: integer` (unique per edition)
  Properties: `name: string, location: string, startDate: date, endDate: date.`
- **Stage** — `:Stage`
  Identifier: `stage_id: integer`

Properties: `name: string, maxCapacity: integer, locationDescription: string.`

- **Performance** — `:Performance`
  Identifier: `performance_id: integer`
  Properties: `title: string, genre: string, startTime: datetime, endTime: datetime.`

- **Solo Musician (ISA child)** — `:Artist:SoloMusician`
  Identifier: `artist_id: integer`
  Inherited: `artist_id, stageName, country, genre`
  Specific: `fullName: string, nationality: string, instrument: string.`

- **Band (ISA child)** — `:Artist:Band`
  Identifier: `artist_id: integer`
  Inherited: `artist_id, stageName, country, genre`
  Specific: `bandName: string, formationYear: integer.`

- **Stand** — `:Stand`
  Identifier: `stand_id: integer`
  Properties: `standName: string, sponsor: string.`

- **Product** — `:Product`
  Identifier: `barcode: integer`
  Properties: `name: string, category: string ('Drink' | 'Food' | 'Merchandise'), price: float.`

- **Ticket** — `:Ticket`
  Identifier: `ticket_id: integer`
  Properties: `type: string ('Regular' | 'DayPass' | 'VIP'), price: float, startValidity: datetime, endValidity: datetime.`

- **Customer** — `:Customer`
  Identifier: `customer_id: integer`
  Properties: `fullName: string, email: string, phone: string.`

1.2.3 Relationships : Definition and semantics

- **R1** — `(:Festival)-[:INCLUDES]->(:Stage)`
  Each festival includes one or more stages where performances take place.

- **R2** — `(:Stage)-[:HAS_PERFORMANCE]->(:Performance)`
  Links each stage to the performances hosted on it during a festival.

- **R3** — `(:Artist)-[:PERFORMS_IN]->(:Performance)`
  Connects artists (either solo musicians or bands) to the performances they take part in.

- **R4** — `(:Festival)-[:HOSTS]->(:Stand)`
  Indicates that a festival hosts several stands.

- **R5** — `(:Stand)-[:SELLS_PRODUCT]->(:Product)`
  Defines which products are sold at each stand.
- **R6** — `(:Customer)-[:BOUGHT_PRODUCT]->(:Product)`
  Represents customers purchasing products from the festival stands.
- **R7** — `(:Customer)-[:PURCHASED_TICKET]->(:Ticket)`
  Records ticket purchases by customers, with additional details such as `paymentDate` and `paymentMethod`, which are properties of this relationship.
- **R8** — `(:Festival)-[:SELLS]->(:Ticket)`
  Shows the link between each festival and the tickets it sells or makes available to customers.

1.2.4 Data loading and database adaptation

- **All node datasets** were imported into Neo4j Aura via **Data Importer** from the CSVs derived from the first SQL project.
  To align with the property-graph model, we adapted the relational data by: renaming headers to follow Neo4j naming conventions (lowercase, no spaces, underscores), converting identifiers from strings to `integers` for consistency and indexing.
- **For the relationships**, we followed the same process: the associative tables from the relational model were adapted into eight CSV files (one per relationship). All foreign keys were aligned to the new integer identifiers to ensure consistency across connected entities.
- **Constraints.** After loading nodes and relationships, we created uniqueness constraints to ensure data integrity and efficient query execution. Each main entity has a unique identifier constraint on its primary key (e.g., `:Artist(artist_id)`, `:Festival(festival_id)`, `:Stage(stage_id)`, `:Performance(performance_id)`, `:Stand(stand_id)`, `:Product(barcode)`, `:Ticket(ticket_id)`, `:Customer(customer_id)`).
- **Artist multi-label ISA.** In Neo4j, the `Artist` superclass is represented using a **multi-label Total and Exclusive (T.E.) hierarchy**.
  Each solo musician node is created with the labels `:Artist:SoloMusician`, and each band node with the labels `:Artist:Band`.
  They share common attributes such as `artist_id`, `stageName`, `country`, and `genre`, while retaining subtype-specific attributes (`instrument`, `nationality` for solo musicians; `bandName`, `formationYear` for bands).
  A **uniqueness constraint** is defined on `:Artist(artist_id)` to ensure global identifier consistency across all artists.

This design allows querying all artists with:

```
MATCH (a:Artist)
RETURN a;
```

Solo Musicians with:

```
MATCH (a:SoloMusician)
RETURN a;
```

and Bands with:

```
MATCH (a:Band)
RETURN a;
```

Diagnostic checks confirm that all nodes respect the Total & Exclusive inheritance structure:

// Total check — every Artist is either a Band or SoloMusician

```
MATCH (a:Artist)
WHERE NOT (a:SoloMusician) AND NOT (a:Band)
RETURN a LIMIT 5;
```

// Exclusive check — no node is both Band and SoloMusician

```
MATCH (a:Artist:SoloMusician:Band)
RETURN a LIMIT 5;
```

// Uniqueness constraint on artist_id

```
CREATE CONSTRAINT artist_id_unique IF NOT EXISTS
FOR (a:Artist)
REQUIRE a.artist_id IS UNIQUE;
```

Verification of multi-label inheritance: The results confirm that all `SoloMusician` and `Band` nodes correctly include the additional `:Artist` label, demonstrating that the ISA (inheritance) structure has been implemented properly in Neo4j.

Verification of multi-label inheritance. The results confirm that all `SoloMusician` and `Band` nodes correctly include the additional `:Artist` label, demonstrating that the ISA (inheritance) structure has been implemented properly in Neo4j.

```
MATCH (a:SoloMusician)
RETURN a.stageName, labels(a);
```

a.stageName	labels(a)
"Elisa"	["Artist", "SoloMusician"]
"Tiziano Ferro"	["Artist", "SoloMusician"]

```
MATCH (b:Band)
RETURN b.stageName, labels(b);
```

b.stageName	labels(b)
"Màneskin"	["Band", "Artist"]
"Negramaro"	["Band", "Artist"]

- **The complete Cypher import scripts**, including **all node definitions**, the **Artist multi-label ISA structure**, and **all relationship creations**, are provided in the **"Cypher Queries"** file.

1.3 Cypher queries

All Cypher queries are included in the **"Query Code"** file. When outputs were too long, only the first five rows are shown here. All related complete CSV files are stored in **"Images and Queries → Query Output"**.

1.4 Create / Update / Delete Queries

1.4.1 1) CREATE – Aurora Luna's debut performance

Creates a new `:Artist:SoloMusician` node for *Aurora Luna* with all related attributes and links her to the existing performance *"Neon Echoes"* through the `:PERFORMS_IN` relationship. It returns the newly created artist node together with the connected performance.

```
MATCH (p:Performance {title: "Neon Echoes"})
CREATE (a:Artist:SoloMusician {
    artist_id: 4051,
    stageName: "Aurora Luna",
    country: "Italy",
    genre: "Electronic",
    fullName: "Aurora Bianchi",
    nationality: "Italian",
    instrument: "synth"
})
CREATE (a)-[:PERFORMS_IN]->(p)
RETURN a, p;
```

Table 1.1: Query-1 Results

a	p
(:SoloMusician:Artist {country: Italy, stageName: Aurora Luna, nationality: Italian, genre: Electronic, fullName: Aurora Bianchi, instrument: synth, artist_id: 4051})	(:Performance {performance_id: 3001, genre: Electronic, startTime: 2025-06-20 21:00, endTime: 2025-06-20 23:30, title: Neon Echoes})

1.4.2 2) CREATE – New Stage "Eco Dome Milano"

Creates a new `:Stage` node named **"Eco Dome Milano"** and connects it to the festival **"Milano Soundwave"** through the `:INCLUDES` relationship. It returns the festival name and stage details (ID, name, capacity, and location), ordered by stage ID.

```
MATCH (f:Festival {name: "Milano Soundwave"})
CREATE (s:Stage {
    stage_id: 2051,
    name: "Eco Dome Milano",
    maxCapacity: 7000,
    locationDescription: "Parco Sempione - area nord"
})
CREATE (f)-[:INCLUDES]->(s)
RETURN f.name AS Festival,
       s.stage_id AS ID_Stage,
       s.name AS Stage,
       s.maxCapacity AS MaxCap,
       s.locationDescription AS Location
ORDER BY ID_Stage;
```

Table 1.2: Query-2 Results

Festival	ID_Stage	Stage	MaxCap	Location
Milano Soundwave	2051	Eco Dome Milano	7000	Parco Sempione - area nord

1.4.3 3) UPDATE – VIP Ticket Price Increase

Increases the price of all `:Ticket` nodes of type **VIP** sold by the festival **"Milano Soundwave"** by 10%. It returns each ticket's ID with its old and new price values, rounded to two decimals.

```
MATCH (:Festival {name: "Milano Soundwave"})-[:SELLS]->(t:Ticket {type: "VIP"})
```

```
WITH t, t.price AS oldPrice
SET t.price = t.price * 1.10
RETURN
    t.ticket_id AS Ticket_ID,
    ROUND(oldPrice, 2) AS OldPrice,
    ROUND(t.price, 2) AS NewPrice;
```

Table 1.3: Query-3 Results

Ticket_ID	OldPrice	NewPrice
9006	199.0	218.9
9004	180.0	198.0
9121	199.0	218.9
9002	399.0	438.9

1.4.4 4) UPDATE – Vegetarian Sandwich on Sale

Applies a **€2 discount** to the product **"Vegetarian Sandwich"**, updating its price value in the database. It returns the product name along with its original and discounted prices (rounded to two decimals).

```
MATCH (p:Product {name: "Vegetarian Sandwich"})
WITH p, p.price AS originalPrice
SET p.price = p.price - 2
RETURN
    p.name AS Product,
    ROUND(originalPrice, 2) AS OriginalPrice,
    ROUND(p.price, 2) AS DiscountedPrice;
```

Table 1.4: Query-4 Results

Product	OriginalPrice	DiscountedPrice
Vegetarian Sandwich	7.0	5.0

1.4.5 5) DELETE – Remove Outdated Festivals (Before 31 Dec 2025)

Deletes all `:Festival` nodes whose start date is before **31 December 2025**, then removes any orphan nodes of types `:Stage`, `:Performance`, `:Stand`, `:Ticket`, and `:Product`. It returns the number and names of the deleted festivals.

```
MATCH (f:Festival)
```

```
WHERE date(f.startDate) < date("2025-12-31")
WITH collect(f) AS toDelete, collect(f.name) AS deletedNames
CALL {
    WITH toDelete
    UNWIND toDelete AS fest
    DETACH DELETE fest
}

CALL {
    // cleanup orphans: Stage, Performance, Stand, Ticket, Product
    WITH 1 AS _
    CALL {
        WITH _
        MATCH (n:Stage)
        WHERE NOT (n)--()
        WITH collect(n) AS ns
        FOREACH (x IN ns | DETACH DELETE x)
    }
    CALL {
        WITH _
        MATCH (n:Performance)
        WHERE NOT (n)--()
        WITH collect(n) AS ns
        FOREACH (x IN ns | DETACH DELETE x)
    }
    CALL {
        WITH _
        MATCH (n:Stand)
        WHERE NOT (n)--()
        WITH collect(n) AS ns
        FOREACH (x IN ns | DETACH DELETE x)
    }
    CALL {
        WITH _
        MATCH (n:Ticket)
        WHERE NOT (n)--()
        WITH collect(n) AS ns
        FOREACH (x IN ns | DETACH DELETE x)
    }
    CALL {
        WITH _
        MATCH (n:Product)
        WHERE NOT (n)--()
        WITH collect(n) AS ns
        FOREACH (x IN ns | DETACH DELETE x)
    }
    RETURN 0 AS _
}
```

```
WITH deletedNames
RETURN
    size(deletedNames) AS Deleted_Festivals_Count,
    deletedNames AS Deleted_Festivals_Names;
```

Table 1.5: Query-5 Results

Deleted_Festivals_Count	Deleted_Festivals_Names
5	[Milano Soundwave, Roma Indie Fest, Firenze Summer Jam, Napoli Beats, Reggio Vibes]

Retrieval queries → **2 queries with at least two nodes in the MATCH statement and conditions**

Retrieval Queries

1.4.6 6) Italian Artists performing at Milano Soundwave

Finds all Italian artists (`country = "Italy"`) who perform in the festival **"Milano Soundwave"** by traversing the `INCLUDES`, `HAS_PERFORMANCE`, and `PERFORMS_IN` relationships. It returns each artist's stage name and genre in alphabetical order.

```
MATCH (f:Festival {name: "Milano Soundwave"})-[:INCLUDES]->
(:Stage)-[:HAS_PERFORMANCE]->(p:Performance)<-[:PERFORMS_IN]-(a:Artist)
WHERE a.country = "Italy"
RETURN DISTINCT a.stageName AS Artist, a.genre AS Genre
ORDER BY Artist;
```

Table 1.6: Query-6 Results

Artist	Genre
Elisa	Pop
Fabrizio Bosso Quartet	Jazz
Negramaro	Pop
Paolo Fresu	Jazz

1.4.7 7) Electronic Performances after 20 June 2025

Retrieves all **Electronic** performances that occur after **20 June 2025** on stages with a maximum capacity greater than **10,000**. It returns the performance ID, title, start time, stage name, and capacity.

```
WITH datetime("2025-06-20T00:00:00") AS fromDate
MATCH (:Festival)-[:INCLUDES]->(s:Stage)-[:HAS_PERFORMANCE]->(p:Performance)
WHERE p.genre = "Electronic"
  AND p.startTime > fromDate
  AND s.maxCapacity > 10000
RETURN
  p.performance_id AS performance_id,
  p.title AS title,
  p.startTime AS start,
  s.name AS stage,
  s.maxCapacity AS capacity
ORDER BY start;
```

Table 1.7: Query-7 Results

performance_id	title	start	stage	capacity
3001	Neon Echoes	2025-06-20 21:00	Main Stage Milano	20000
3017	Ocean Frequencies	2026-09-05 21:00	Verona Rock Arena	16000

→ 2 queries with at least two nodes in the MATCH statement, conditions and aggregation without a WITH statement

1.4.8 8) Top 5 Most Active Stands

Counts the number of products sold by each stand through the `SELLS_PRODUCT` relationship to identify the most active ones. It returns the **top 5 stands** ranked by total products sold.

```
MATCH (s:Stand)-[:SELLS_PRODUCT]->(prod:Product)
RETURN s.standName AS Stand, COUNT(prod) AS ProductsSold
ORDER BY ProductsSold DESC
LIMIT 5;
```

Table 1.8: Query-8 Results

Stand	ProductsSold
Street Pizza	2
Burger House	2
Heineken Beer Bar	1
Trieste Beer Garden	1
Beer Street Verona	1

1.4.9 9) Ticket Economy at Roma Indie Fest

Calculates the average ticket price per ticket type for the festival **"Roma Indie Fest"**, along with the total number of tickets sold. It returns ticket type, average price, and ticket count, ordered by descending average price.

```
MATCH (f:Festival {name:"Roma Indie Fest"})-[:SELLS]->(t:Ticket)
RETURN t.type AS type,
       round(avg(t.price), 2) AS avgPrice,
       count(*) AS tickets
ORDER BY avgPrice DESC;
```

→ 2 queries with at least two nodes in the MATCH statement, conditions and one WITH statement

1.4.10 Query 9 Output

Table 1.9: Query-9 Results

Ticket_Type	AvgPrice	N_Tickets
VIP	199.42	3
REGULAR	146.59	4
DAY_PASS	75.5	1

1.4.11 10) Food & Drink Fans

Identifies customers who have purchased both **food** and **drink** products by combining two subqueries with a `WITH` clause. It returns the customer's full name along with the lists of food and drink items they bought.

```
MATCH (c:Customer)-[:BOUGHT_PRODUCT]->(pf:Product)
WHERE pf.category = "Food"
WITH c, COLLECT(pf.name) AS foods
MATCH (c)-[:BOUGHT_PRODUCT]->(pd:Product)
WHERE pd.category = "Drink"
RETURN c.fullName AS Customer,
       foods AS Food_Products,
       COLLECT(pd.name) AS Drink_Products;
```

Table 1.10: Query-10 Results

Customer	Food_Products	Drink_Products
Sofia Fabbri	[Classic Burger]	[Coca-Cola Zero 330ml]
Guendalina Nardi	[Cheese Fries]	[Gin Tonic, Campari Spritz]
Sofia Varotto	[Caesar Salad]	[Orange Juice]

1.4.12 11) Frequent Buyers

Finds customers who have purchased **two or more distinct tickets**, counting each unique ticket type per customer using a `WITH` clause. It returns the customer ID, full name, and total number of tickets purchased, ordered by highest count.

```
MATCH (c:Customer)-[:PURCHASED_TICKET]->(t:Ticket)
WITH c, count(DISTINCT t) AS bought
WHERE bought >= 2
RETURN c.customer_id AS customer_id,
       c.fullName AS customer,
       bought AS ticketsBought
ORDER BY ticketsBought DESC, customer;
```

Table 1.11: Query-11 Results

customer_id	customer	ticketsBought
6019	Alessandro Bruno	2
6014	Alessia De Luca	2
6009	Andrea Conti	2
6031	Antonio Fiore	2

→ 2 queries with at least three nodes in the MATCH statement, conditions and multiple WITH statements

1.4.13 12) Festival Tourers

Detects artists who performed in **two or more different festivals** by collecting distinct festival names per artist through multiple `WITH` statements. It returns the artist's name, total number of festivals, and the list of attended festivals.

```
MATCH (a:Artist)-[:PERFORMS_IN]->(p:Performance)
MATCH (p)<-[:HAS_PERFORMANCE]-(:Stage)<-[:INCLUDES]-(f:Festival)
WITH a, COLLECT(DISTINCT f.name) AS festivals
WITH a, festivals, SIZE(festivals) AS numFest
```

```
WHERE numFest >= 2
RETURN a.stageName AS Artist,
       numFest AS Festivals,
       festivals
ORDER BY numFest DESC;
```

Table 1.12: Query-12 Results

Artist	Festivals	festivals
David Guetta	8	[Roma Indie Fest, Napoli Beats, Bari Music Garden, Palermo Sunset Fest, Padova Indie Days, Rimini Beach Beats, Lucca Music Days, Como Lakeside Fest]
Paolo Conte Trio	8	[Roma Indie Fest, Palermo Sunset Fest, Cagliari Wave, Trieste Electro Sound, Ancona Pop Nights, Rimini Beach Beats, Reggio Vibes, Pescara Festival del Mare]
Fabrizio Bosso Quartet	7	[Milano Soundwave, Verona Rock Nights, Lecce Live Energy, Trieste Electro Sound, Ancona Pop Nights, Rimini Beach Beats, Reggio Vibes]
Paolo Fresu	6	[Milano Soundwave, Verona Rock Nights, Cagliari Wave, Lecce Live Energy, Ancona Pop Nights, Reggio Vibes]

1.4.14 13) Longest Performance per Festival

Computes the **duration** of each performance by comparing start and end times, then identifies the longest one per festival. It returns the festival name, title of the longest performance, and its duration in minutes, sorted by duration.

```
MATCH (f:Festival)-[:INCLUDES]->(:Stage)-[:HAS_PERFORMANCE]->(p:Performance)
WHERE p.startTime IS NOT NULL AND p.endTime IS NOT NULL
WITH f, p, duration.between(p.startTime, p.endTime) AS d
WITH f, p, (toInteger(d.seconds) / 60) AS mins
ORDER BY f.name, mins DESC, p.title
WITH f, collect({title: p.title, mins: mins})[0] AS top
RETURN f.name AS festival,
       top.title AS longestPerformance,
       round(top.mins) AS minutes
ORDER BY minutes DESC, festival;
```

Table 1.13: Query-13 Results

festival	longestPerformance	minutes
Genova Sea Sounds	Dancing Mirage	480
Padova Indie Days	Electric Bloom	480
Bologna Groove Fest	Crimson Horizon	390
Verona Rock Nights	Summer Groove	390

→ 1 query with variable-length paths in the MATCH statement and conditions

1.4.15 14) Chasing Artistic Connections: Variable-Lenght Path Between Artists

Explores variable-length paths (2 to 10 hops) connecting *Laura Pausini* and *Marco Mengoni* through shared performances via the `:PERFORMS_IN` relationship. It returns one sample path showing their indirect artistic connections.

```
MATCH (a1:Artist {stageName: "Laura Pausini"}),
      (a2:Artist {stageName: "Marco Mengoni"})
MATCH path = (a1)-[:PERFORMS_IN*2..10]-(a2)
RETURN path
LIMIT 1;
```

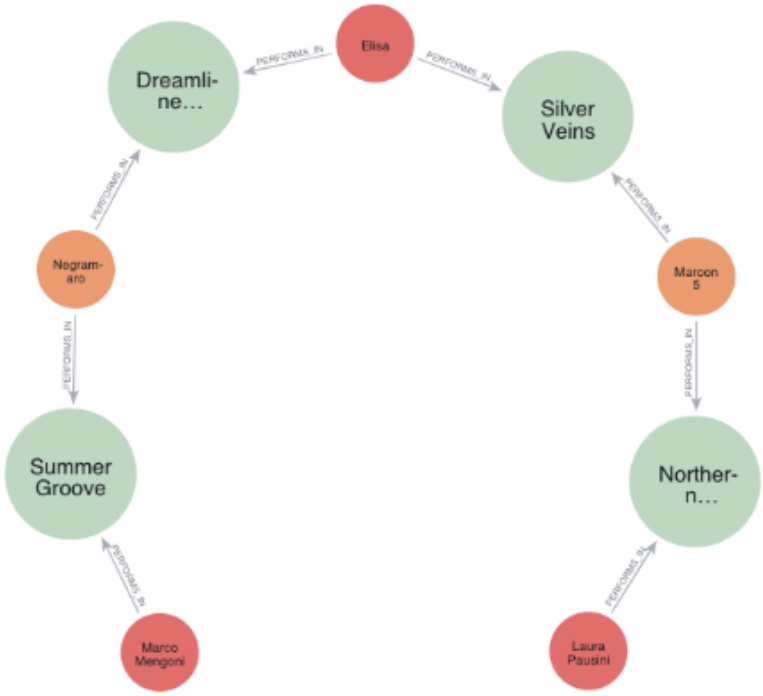


Figure 1.1: Query-14 Results

→ **1 query that computes the shortest path**

1.4.16 15) Closest Connection: Shortest Path Between Two Artists

Finds the **shortest path** connecting *Laura Pausini* and *Marco Mengoni*, considering only the `:PERFORMS_IN` relationships alternating between artists and performances. It returns the shortest path found in the graph.

```
MATCH (a1:Artist {stageName: "Laura Pausini"}),
      (a2:Artist {stageName: "Marco Mengoni"})
MATCH p = shortestPath((a1)-[:PERFORMS_IN*2..10]-(a2))
RETURN p;
```

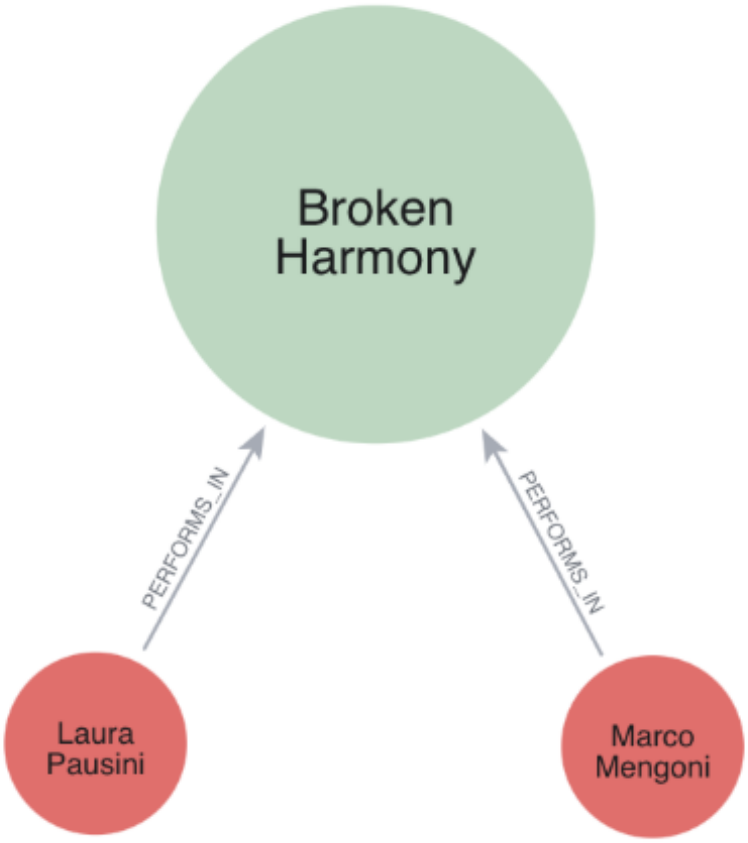


Figure 1.2: Query-15 Results

Chapter 2

BPMN

2.1 BPMN diagram representation

Provided text: *The StoreStore Company is interested in developing an e-commerce system for their electronic stores, and asked you to provide them with a BPMN diagram depicting the pipeline of interactions with the customer, and the preparation and shipping of an order. The process starts with the customer asking for the available products. The customer service receives the request, and forwards it to the inventory management, which retrieves the list of available products. After receiving the list, the customer prepares its order and sends it to the customer service. The preparation of the order is handled by the inventory management, which checks the availability of each product and, when available, handles its packaging. If any product is not in stock, the customer service sends a notification to the customer, which is required to update the shopping list. Additionally, to allow better management of the warehouse stocks, when a product is running out (i.e., less than 10 items are available), the inventory management places an order to an external supplier before continuing with the order preparation. After all products of the shopping list have been packaged, the customer service notifies the customer that the order is ready for shipping, and then the order is physically sent by the logistics department. Since it is possible that an order may contain defective products, customers are allowed to return items to the customer service. If no item is returned within a week, the order is recorded and the process terminates. Otherwise, the customer service writes a report for the returned items and the process terminates.*

2.1.1 Designed diagram

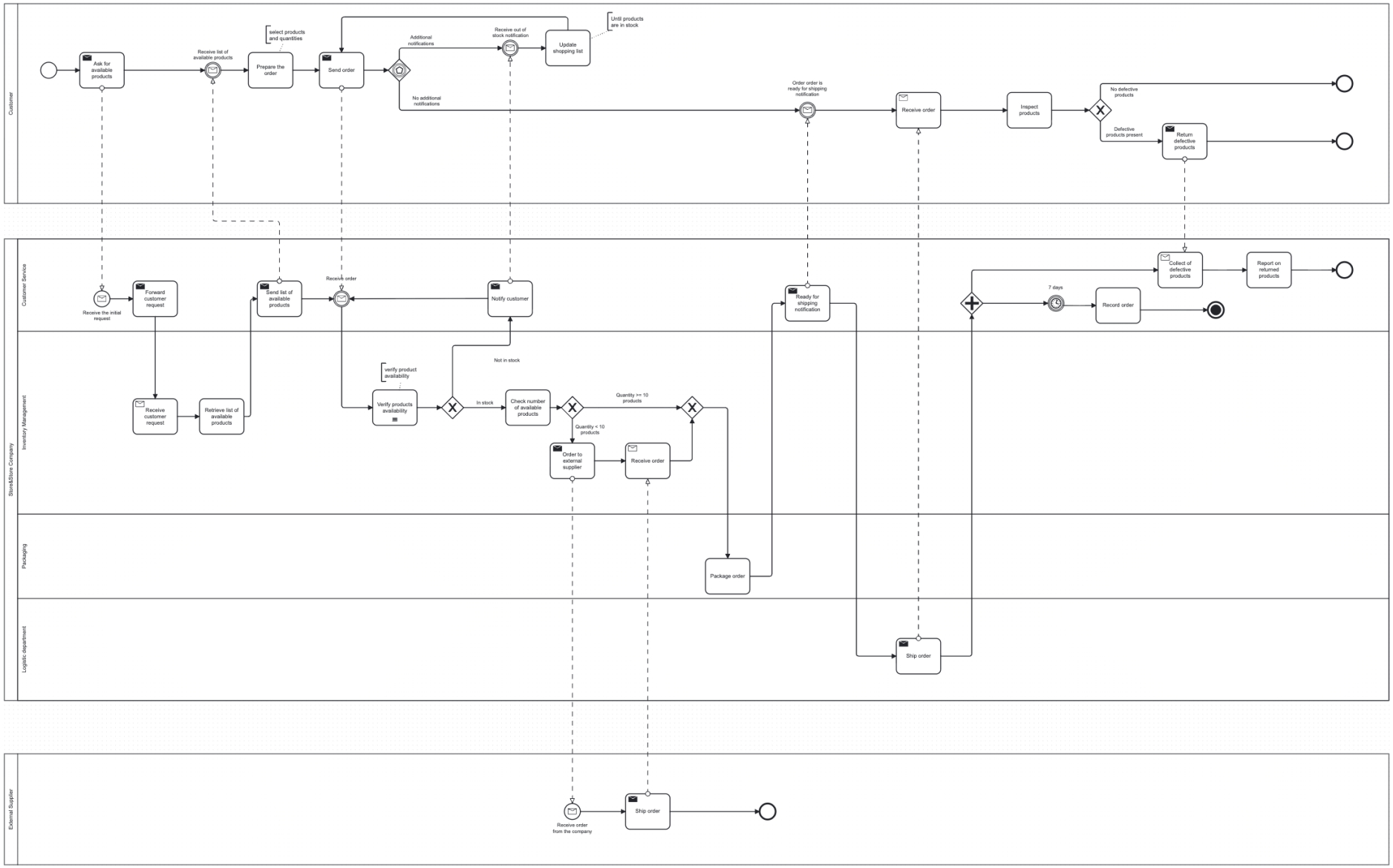


Figure 2.1: Designed BPMN 2.1

Assumptions

A main assumption in this BPMN model concerns the iterative management of product availability and the continuous interaction between the **Customer** and the **Company** until all products become available.

When the **Inventory Management** verifies the stock status, the process is directed through an **exclusive gateway (XOR)**:

Unavailable products

If one or more products are not available, the **Inventory Management** triggers the *"Not in stock"* branch. This leads the **Customer Service** to send an *"Out of Stock Notification"* to the Customer through a **Send Message Task**.

In the meanwhile, the Customer flow continues with an **event-based gateway**, which waits for one of two possible events:

- the reception of an *"Out of Stock Notification"*, allowing the customer to update and resend the order, or
- the reception of an *"Order ready for shipping notification"*.

If the *Out of Stock Notification* event occurs, the Customer performs the task *"Update shopping list"* and sends the updated order back to the **Customer Service** using a **Send Message Task**. On the company side, the **Customer Service** uses an **Intermediate Message Catch Event** (*"Receive order"*) to wait for the customer's updated order. Once the message arrives, the flow returns to the *"Verify products availability"* task in **Inventory Management**, restarting the checking process.

This loop can repeat an indefinite number of times, allowing the customer to modify and resend the order until all requested products are available.

Available products

If all products are available, the **exclusive gateway** routes the company's flow to the *"Check number of available products"* task, after which the process continues toward packaging and shipment. At the same time, the Customer's event-based gateway detects the *"Order ready for shipping notification"*, allowing the customer's flow to proceed accordingly.

This BPMN model enables a flexible, event-driven interaction between the **Customer** and the **Company**, allowing the order to be updated and resent multiple times until all products are available.

2.2 BPMN & Euphoric Events

Euphoric Events: Ticket Purchase and Festival Access Process

The *Euphoric Events* company is responsible for organizing multiple music festivals across Italy and aims to provide an efficient digital system to manage the purchase, validation, and use of event tickets.

The process starts with the customer asking for information about available festivals and ticket types. The customer service receives the request and forwards it to the ticketing system, which checks ticket and festival availability in the company's relational database (RDBMS). The RDBMS stores all relevant data through several main entities: **Festival** (*Name, Location, Start_Date, End_Date*), **Ticket** (*ID_Ticket, Type, Price, Start_Validity, End_Validity, ID_Festival*), **Customer** (*ID_Customer, Full_Name, Email, Phone*), and **Ticket_Payment** (*Ticket_ID, ID_Customer, Payment_Date, Method*). These tables ensure the correct association between each customer, the purchased ticket, and the related festival event.

If no tickets are available, the customer service notifies the customer of the unavailability, and the process terminates. Otherwise, the ticketing system retrieves the available options and sends them back to the customer service, which forwards the ticket catalog to the customer.

After receiving the list, the customer selects the desired festival and ticket type and proceeds with the payment. The customer provides the credit card information to the credit card company for authorization. The external company verifies the payment details and returns the result to the customer. If the payment is declined, the customer is allowed to retry the operation. If the payment is accepted, the transaction is confirmed and the order is sent.

Once the order is received, the ticketing system records the transaction in the RDBMS, updating the **Ticket_Payment**. The system then generates a digital ticket and sends it to the customer. The customer receives the digital ticket, which will later be used to access the festival.

When the festival day arrives, the access control system automatically validates all issued tickets by cross-checking them with the information stored in the RDBMS.

The process terminates either when the ticket purchase fails due payment rejection, or when the customer successfully accesses the festival.

2.2.1 Designed diagram

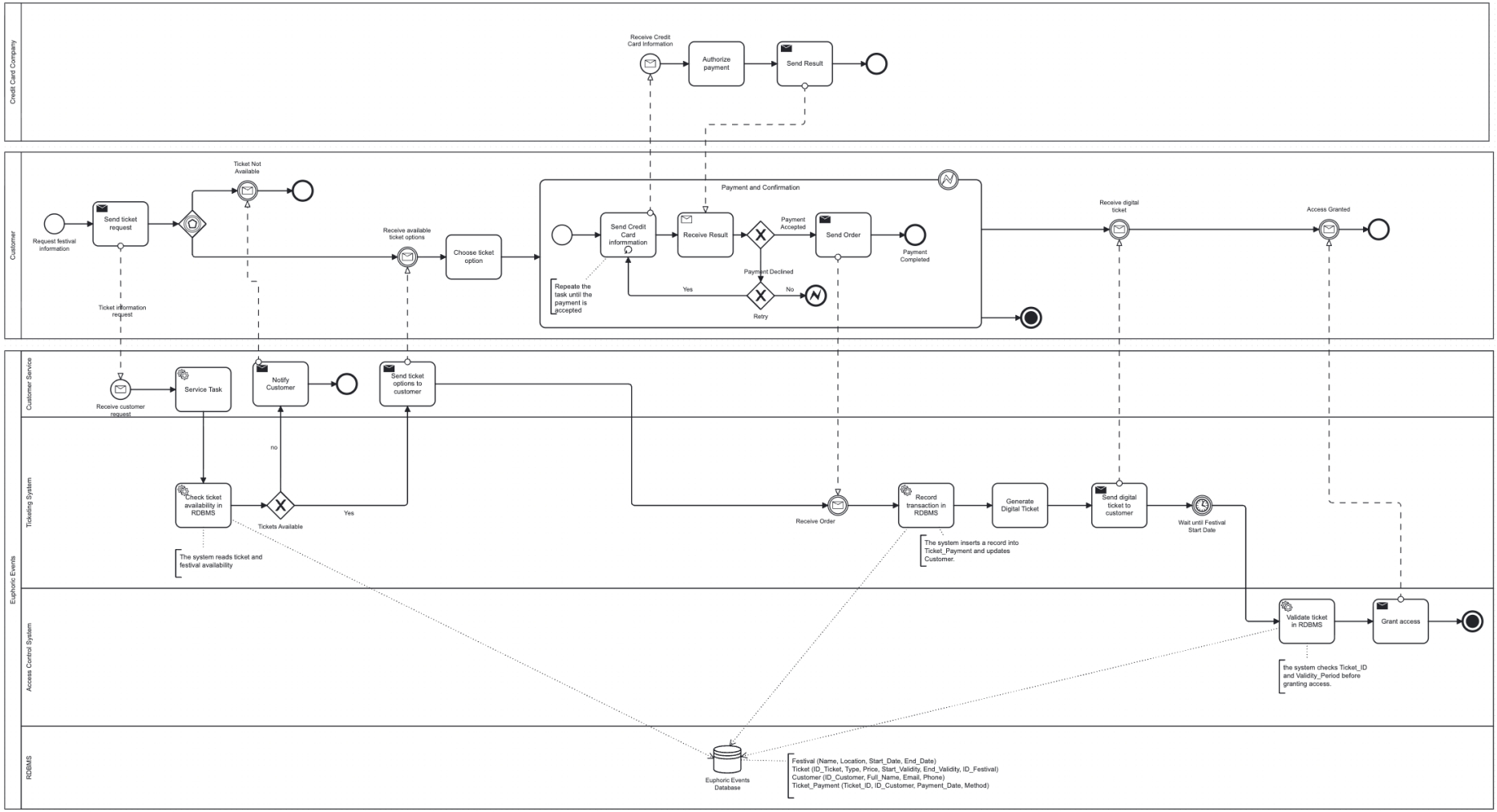


Figure 2.2: Designed BPMN 2.2